Methodology

Methodology I

This section describes the implementation methodology, explicitly detailing how the optimization algorithms are constructed, validated, and analyzed using the Generalized Research Template's `infrastructure` and `tests` ecosystems.

Algorithm Implementation I

Algorithm Implementation II

Gradient Descent Algorithm

The core algorithm implements the iterative procedure for unconstrained optimization. The `optimizer` module uses the standard-library `logging` logger for optional verbose diagnostics; the analysis orchestrator uses `infrastructure.core.logging.utils.get_logger`. Tests run under the hermetic boundaries defined in the test configuration.

Algorithm — Gradient Descent (implemented in the optimizer module)

Input: *Initial point x_0, step size α, tolerance ϵ, max iterations $N_{\max}$*

1. *Initialize $k \leftarrow 0$*
2. ***While*** $k < N_{\max}$ ***do:***
   - *Compute gradient $g_k = \nabla f(x_k)$*
   - ***If*** $\|g_k\|_2 < \epsilon$ ***then return*** x_k *(converged)*
   - *Update: $x_{k+1} \leftarrow x_k - \alpha \cdot g_k$*
   - $k \leftarrow k + 1$
3. ***Return*** x_k *(max iterations reached)*

Infrastructure Integration I

The methodology explicitly bridges theoretical mathematics with production-grade validation through the `infrastructure.scientific` module.

Numerical Stability Analysis

Rather than writing ad-hoc validation code, the project imports `infrastructure.scientific.stability.check_numerical_stability`. This utility subjects the objective function to a barrage of extreme inputs (NaN, Inf, $\pm 10^{10}$) to calculate a formalized stability score. If this score degrades, the analysis orchestrator execution deliberately aborts, ensuring the methodology cannot enter unrecoverable states.

Infrastructure Integration II

Performance Benchmarking

Computational complexity is evaluated not just theoretically, but empirically via `infrastructure.scientific.benchmarking.benchmark_function`. This module captures high-resolution execution timings and memory footprints across dimensionality sweeps, guaranteeing that the $O(n)$ space-time complexity predictions hold true on the host architecture.

Convergence Analysis I

For quadratic functions $f(x) = \frac{1}{2}x^T Ax - b^T x$ where A is positive definite, the convergence factor becomes [@bertsekas1999nonlinear]:

$$\rho = \frac{|\lambda_{\max} - \alpha\lambda_{\min}|}{|\lambda_{\min} + \alpha\lambda_{\max}|} \tag{1}$$

Optimal convergence occurs when $\alpha = \frac{2}{\lambda_{\min}+\lambda_{\max}}$, yielding $\rho = \frac{\kappa-1}{\kappa+1}$.

Experimental Setup I

Step Size Analysis

Step sizes are not chosen ad hoc in the manuscript: they are read from `experiment.step_sizes` in `manuscript/config.yaml` and passed through `run_convergence_experiment()` in `optimizati on_analysis.py`. The active grid for this build is:

- $\alpha = 0.01$ (conservative)
- $\alpha = 0.1$ (conservative)
- $\alpha = 0.5$ (near-optimal)
- $\alpha = 1.0$ (near-optimal)
- $\alpha = 1.5$ (aggressive)
- $\alpha = 2.5$ (divergent (expected unstable for H = I))

Labels follow the same agency taxonomy used for plot colours (`_agency_category` in the analysis script): **conservative** (small α, slow but stable on $H = I$), **near-optimal** (including $\alpha = 1$ where the method reaches x^* in one step for this quadratic), **aggressive** ($1 < \alpha < 2$, still linearly contracting in $|x - x^*|$ but oscillatory in sign), and **divergent** ($|1 - \alpha| \geq 1$).

Experimental Setup II

Zero-Mock Testing Methodology

The most critical aspect of the project's methodology is its validation framework. The project is governed by a strict Zero-Mock testing policy, evaluated actively by executing `uv run pytest projects/template_code_project/tests/` during the infrastructure build phase.

1. **Project tests**: `projects/template_code_project/tests/test_optimizer.py` holds **52** collected tests that exercise `src/optimizer.py` (typical, edge, boundary, and pathological inputs including NaN/Inf and zero gradients) and, when infrastructure imports succeed, call into `optimization_analysis.py` helpers—without mocks.
2. **Infrastructure validation**: The repository-level `tests/infra_tests/` suite validates shared template modules (e.g. pipeline and discovery helpers) independently of this project's manuscript.
3. **Coverage Gates**: The GitHub Actions CI workflow enforces a mandatory 90% statement coverage gate on `projects/template_code_project/src/` prior to treating the project as

Analysis Pipeline & LaTeX Integration I

The automated analysis script leverages `infrastructure.core.progress` (`ProgressBar`, `SubStageProgress`) to orchestrate experiments, collect convergence trajectories, and generate publication-quality visualizations seamlessly.

The research template supports advanced LaTeX customization through the `preamble.md` configuration. This is ingested directly by `infrastructure.rendering.latex_utils.py` and `pdf_renderer.py`, automatically linking compiled PGF plots and BibTeX citations. This automated approach ensures an unbreakable chain of custody from raw algorithmic execution to the final rendered manuscript.